

딥러닝 파이토치 교과서 5장 - Part 1 (~5.2)

📅 날짜	@2026년 3월 24일
👤 발표팀	3팀
⚙️ 상태	시작 전
🏷️ 태그	3주차

5.1 합성곱 신경망

→ 이미지나 영상 처리하는 데 유용함. 왜?

보통 3x3이면 픽셀을 펼쳐서 하느라 데이터의 공간적 구조를 무시함. 그래서 합성곱층이 도입.

합성곱층과 풀링층을 거치면서 입력 이미지의 주요 특성 벡터를 추출. 완전연결층을 거치면서 1차원 벡터로 변환됨. → 소프트맥스 함수를 활용해 최종 결과 출력

1. 입력층: 데이터가 최초로 거치게되는 계층.

이미지는 높이, 너비, 채널의 값을 갖는 3차원 데이터임. 채널은 컬러면 3, 흑백이면 1.

2. 합성곱층: 데이터에서 특성을 투철하는 역할을 수행. 입력 이미지가 들어왔을 때 커널이나 필터를 사용. 모든 영역을 훑으면서 특성을 추출하는데 이를 특성 맵이 됨.

보통 3x3, 5x5로 움직이면서 이동하고 스트라이드라는 지정된 간격에 맞춰서 움직이면서 특성 추출.

그레이스케일일 때,

▼ 그림 5-4 입력 이미지에 3×3 필터 적용

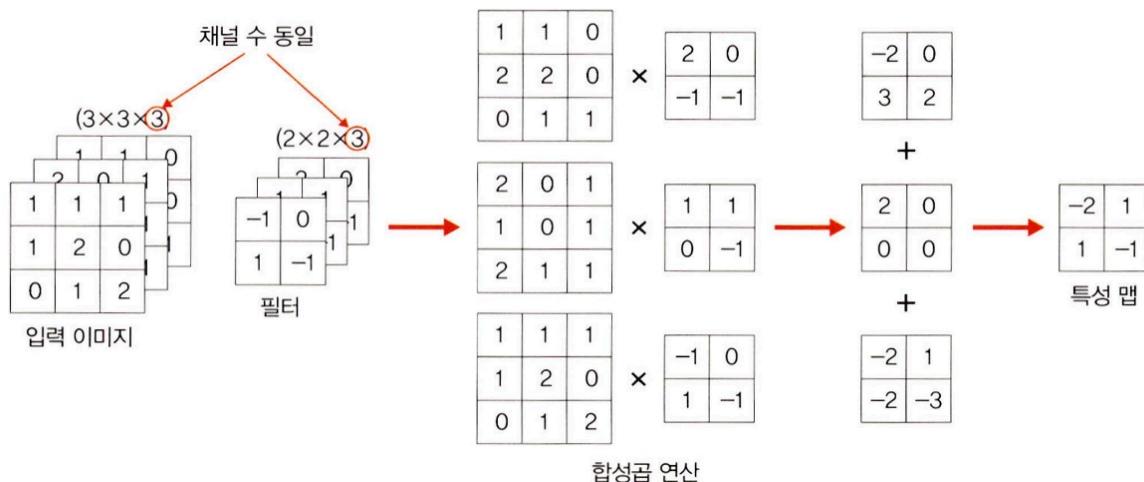


2단계. 필터가 1만큼 이동

$$(0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) = 1$$

스트라이드가 1이라 1만큼씩 이동한 것
컬러 이미지일 때,

▼ 그림 5-10 컬러 이미지 합성곱



필터는 1개지만 필터 채널이 3임. rgb 따로따로 해야해서 서로 다른 가중치로 합성곱 적용 후 결과 계산.

하이퍼파라미터는 필터 개수, 필터 크기, 스트라이드, 패딩(늘려주는 거)

2. 풀링층

합성곱층과 유사하게 특성 맵의 차원을 다운 샘플링하여 연산량을 감소시킴 → 학습 효과 증대

- 최대 풀링: 대상 영역에서 최댓값 추출

- 평균 풀링: 대상 영역에서 평균을 반화

보통은 최대풀링 사용 → 평균 풀링은 중요한 가중치를 갖는 값의 특성이 희미해질 수 있어서.

선택된 값 안에서 그냥 최대값 선택 혹은 평균 구해서 선택 후 스트라이드 만큼 이동함.

그래서 하이퍼파라미터는 필터 크기와 스트라이드.

3. 완전연결층

차원이 축소된 특성 맵을 받아서 3차원 벡터에서 1차원 벡터로 펼치는 역할을 수행.

4. 출력층

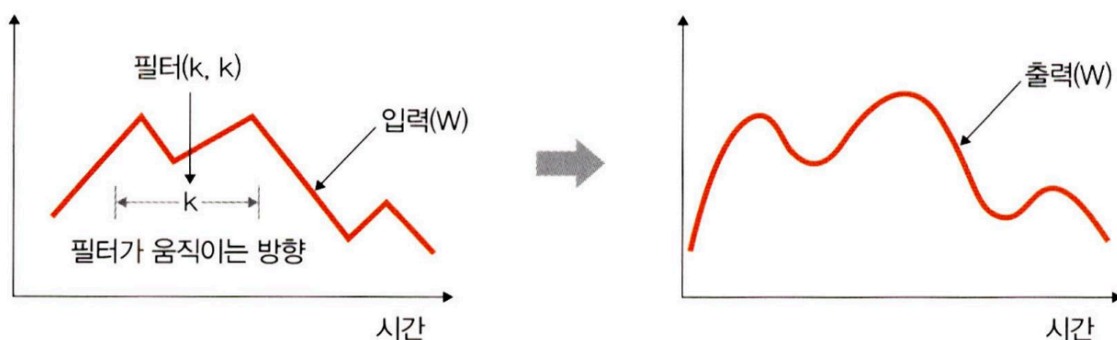
소프트맥스 활성화 함수 사용. 입력받은 값을 0~1사이의 값으로 출력. 따라서 마지막 출력층의 소프트맥스 함수를 이용하여 이미지가 각 레이블에 속할 확률 값을 출력해서 이를 최종 값으로 선정함.

합성곱의 종류: 이동하는 방향의 수와 출력의 형태에 따라 1D, 2D, 3D로 분류할 수 있음.

1. 1D

필터가 시간을 축으로 좌우로만 이동할 수 있는 합성곱을 말함. 그대로 움직이기 때문에 입력과 출력이 같음. 단 완곡하게 만드는 역할

▼ 그림 5-19 1D 합성곱

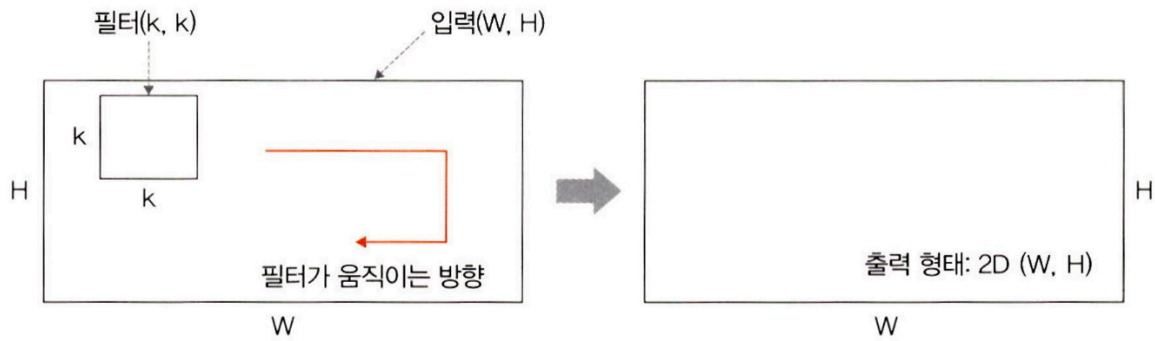


- 입력: W 너비(Width)
- 필터: $k \times k$ (높이 $\times$ 너비)
- 출력: W 너비(Width)

2. 2D

필터가 두 개의 방향으로 움직이는 형태, 입력과 필터에 대한 출력이 (W,H) 가 돼서 출력 형태 자체가 2D 행렬이 됨.

♥ 그림 5-20 2D 합성곱

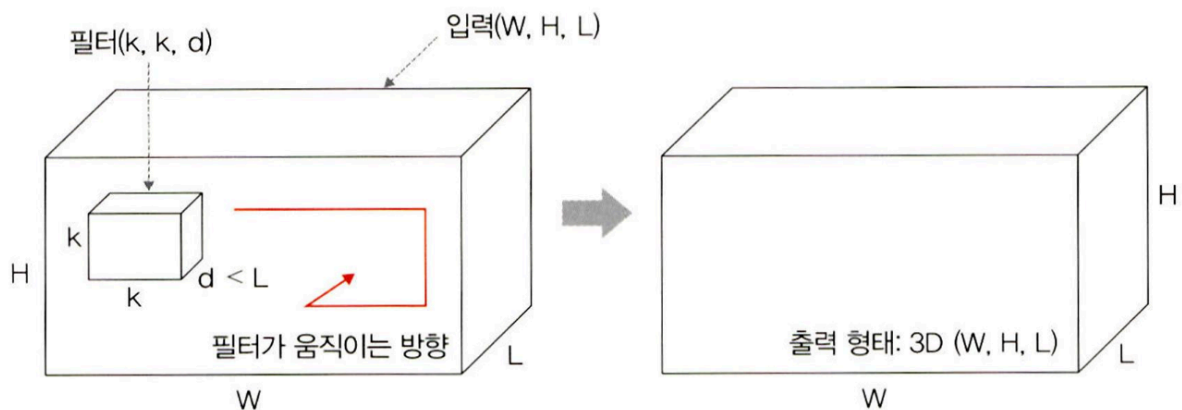


- 입력: W 너비(Width), H 높이(Height)
- 필터: $k \times k$ (높이 $\times$ 너비)
- 출력: W 너비(Width), H 높이(Height)

3. 3D 합성곱

움직이는 방향이 세 개가 있음. 따라서 출력층이 (W,H,L) 임.

♥ 그림 5-21 3D 합성곱



- 입력: W 너비(Width), H 높이(Height), L 길이(Length)
- 필터: $k \times k \times d$ (높이 $\times$ 너비), d: 깊이(depth)
- 출력: W 너비(Width), H 높이(Height), L 길이(Length)

4. 3D 입력을 갖는 2D 합성곱

필터에 대한 길이가 입력 채널의 길이와 같아야 하기 때문에 이런 합성곱 형태가 만들어짐.

출력이 2D 처럼 W,H 로 나옴

사례로는 lenet 이랑 vgg가 있음.

5. 1X1 합성곱

3D 형태로 입력되는데 필터가 (1,1,1) 이라 출력값이 2D로 나옴. 때문에 연산량이 감소되는 효과가 있음. 따라서 GoogleLenet 이 있음.

5.2 코드 실습

GPU 사용을 권장함. 일반적으로 하나와 여러개에 따라 코드가 달라짐

```
#하나인 경우
device = torch.device("cuda:0" if torch.cuda.is_available
() else "cpu")
model = Net()
model.to(device)

#다수인 경우
device = torch.device("cuda" if torch.cuda.is_available()
else "cpu")
model = Net()
if torch.cuda.device_count() > 1:
    model = nn.DataParallel(model)
model.to(device)
```

파이토치의 근간은 C++임 따라서 객체지향 프로그래밍 기반이라는 것을 알아야 함

▼ 표 5-1 nn.xx와 nn.functional.xx의 사용 방법 비교

구분	nn.xx	nn.functional.xx
형태	nn.Conv2d: 클래스 nn.Module 클래스를 상속받아 사용	nn.functional.conv2d: 함수 def function (input)으로 정의된 순수한 함수
호출 방법	먼저 하이퍼파라미터를 전달한 후 함수 호출을 통해 데이터 전달	함수를 호출할 때 하이퍼파라미터, 데이터 전달
위치	nn.Sequential 내에 위치	nn.Sequential에 위치할 수 없음
파라미터	파라미터를 새로 정의할 필요 없음	가중치를 수동으로 전달해야 할 때마다 자체 가중치 를 정의

사용하는 위치가 다름.